Table 1: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means the denominator is faster; **bold** marks >1.1×.

Program	**ADT**	**SoA**	**End-to-end**					**Fold passes**					**Map passes**				
	fields	bufs	Am (s)	Ai (s)	Sm (s)	Am/Sm	Ai/Sm	Am (s)	Ai (s)	Sm (s)	Am/Sm	Ai/Sm	Am (s)	Ai (s)	Sm (s)	Am/Sm	Ai/Sm
OctTree	16	9	0.6715	–	0.6716	1.00×	–	0.3685	–	0.3236	**1.14×**	–	0.3030	–	0.3481	0.87×	–
Compiler	9	8	0.2847	1.086	0.2143	**1.33×**	**5.07×**	0.1258	0.7166	0.0389	**3.23×**	**18.42×**	0.1589	0.3696	0.1753	0.91×	**2.11×**
DBQuery	15	13	0.2579	0.3267	0.2503	1.03×	**1.31×**	0.1217	0.1753	0.0851	**1.43×**	**2.06×**	0.1362	0.1514	0.1651	0.82×	0.92×
DecisionTree	7	6	3.726	5.579	2.934	**1.27×**	**1.90×**	3.726	5.579	2.934	**1.27×**	**1.90×**	–	–	–	–	–
DomTree	15	14	0.1969	0.3377	0.1412	**1.39×**	**2.39×**	0.1385	0.2774	0.0615	**2.25×**	**4.51×**	0.0584	0.0603	0.0797	0.73×	0.76×
KDTree	17	16	0.1802	0.2462	0.1399	**1.29×**	**1.76×**	0.1802	0.2462	0.1399	**1.29×**	**1.76×**	–	–	–	–	–
LinearListReduction	11	11	0.0298	0.1318	0.0049	**6.03×**	**26.67×**	0.0298	0.1318	0.0049	**6.03×**	**26.67×**	–	–	–	–	–
reduceNestedList	6	2	0.0853	0.0929	0.0074	**11.58×**	**12.61×**	0.0853	0.0929	0.0074	**11.58×**	**12.61×**	–	–	–	–	–
List	2	2	0.3828	–	0.4496	0.85×	–	0.1321	–	0.1143	**1.16×**	–	0.2508	–	0.3353	0.75×	–
MonoTree	3	2	0.0631	0.1403	0.0645	0.98×	**2.18×**	0.0172	0.0801	0.0223	0.77×	**3.60×**	0.0459	0.0602	0.0422	1.09×	**1.43×**
ObjectGraph	5	4	0.2337	0.4804	0.2419	0.97×	**1.99×**	0.0911	0.3060	0.0807	**1.13×**	**3.79×**	0.1426	0.1744	0.1612	0.88×	1.08×
PiecewiseFunctions	8	7	0.3741	0.5545	0.3553	1.05×	**1.56×**	0.1365	0.2932	0.0956	**1.43×**	**3.07×**	0.2376	0.2613	0.2598	0.91×	1.01×
TernaryTree	5	3	0.1153	0.1432	0.1042	**1.11×**	**1.37×**	0.0294	0.0501	0.0267	1.10×	**1.87×**	0.0859	0.0931	0.0775	**1.11×**	**1.20×**
Trie	10	9	0.2168	0.2939	0.1990	1.09×	**1.48×**	0.0642	0.1332	0.0324	**1.99×**	**4.12×**	0.1525	0.1607	0.1666	0.92×	0.96×
ColorOctree	25	18	0.0876	0.1172	0.0738	**1.19×**	**1.59×**	0.0876	0.1172	0.0738	**1.19×**	**1.59×**	–	–	–	–	–

Table 2: Native PAPI metrics speedup summary across all passes. For each program and metric, speedup is AoS/SoA where each side is the sum of per-pass median counter values. >1× means SoA reports fewer events.

Program	**CYC (Am/Sm)**	**INS (Am/Sm)**	**L1D (Am/Sm)**	**L1I (Am/Sm)**	**L2D (Am/Sm)**	**L2I (Am/Sm)**	**LLC (Am/Sm)**
OctTree	1.04×	0.75×	**1.59×**	0.43×	0.24×	0.61×	**3.58×**
Compiler	**1.63×**	0.52×	**4.13×**	0.80×	**14.91×**	0.30×	**6.58×**
DBQuery	1.09×	0.47×	**1.44×**	0.42×	**9.02×**	0.37×	**6.28×**
DecisionTree	**1.27×**	0.59×	**18.42×**	**2.86×**	1.04×	**3.06×**	**36.95×**
DomTree	**1.53×**	0.57×	**2.92×**	0.75×	**21.62×**	0.17×	**10.41×**
KDTree	**1.28×**	0.77×	**1.11×**	0.99×	0.23×	1.00×	**4.63×**
LinearListReduction	**6.06×**	0.90×	**33.77×**	**6.30×**	**38.33×**	**5.73×**	**11.62×**
reduceNestedList	**11.63×**	0.62×	**6.34×**	**2.96×**	**63.44×**	**2.87×**	**6.18×**
List	0.82×	0.80×	**3.12×**	0.44×	0.09×	0.47×	**1.45×**
MonoTree	0.98×	0.85×	0.47×	**1.14×**	0.27×	0.57×	**1.23×**
ObjectGraph	0.95×	0.75×	**3.86×**	0.79×	0.57×	0.77×	**2.91×**
PiecewiseFunctions	1.09×	0.60×	**2.73×**	**1.15×**	**2.01×**	0.48×	**4.45×**
TernaryTree	**1.17×**	0.79×	0.28×	0.56×	0.24×	**2.31×**	**1.39×**
Trie	**1.13×**	0.56×	**3.63×**	**1.13×**	**3.59×**	0.09×	**6.32×**
Geomean	**1.52×**	0.67×	**2.80×**	1.04×	**1.93×**	0.73×	**4.84×**

Table 3: Native PAPI metrics totals across all passes (baseline variants). Each entry is (Am/Sm), where Am and Sm are sums of per-pass median counter values.

Program	**CYC (Am/Sm)**	**INS (Am/Sm)**	**L1D (Am/Sm)**	**L1I (Am/Sm)**	**L2D (Am/Sm)**	**L2I (Am/Sm)**	**LLC (Am/Sm)**
OctTree	2.6e+09/**2.5e+09**	**8.2e+09**/1.1e+10	1.6e+07/**9.8e+06**	**4.2e+05**/9.8e+05	**5.9e+04**/2.5e+05	**1.8e+04**/3.0e+04	6.4e+07/**1.8e+07**
Compiler	9.4e+08/**5.8e+08**	**9.9e+08**/1.9e+09	1.3e+07/**3.1e+06**	**2.1e+05**/2.7e+05	1.2e+06/**8.4e+04**	**1.0e+04**/3.4e+04	5.1e+07/**7.7e+06**
DBQuery	9.4e+08/**8.7e+08**	**1.1e+09**/2.2e+09	4.1e+06/**2.8e+06**	**1.3e+05**/3.2e+05	8.5e+05/**9.4e+04**	**7.2e+03**/1.9e+04	2.1e+07/**3.3e+06**
DecisionTree	1.9e+10/**1.5e+10**	**4.0e+10**/6.8e+10	5.4e+08/**2.9e+07**	1.0e+05/**3.6e+04**	2.2e+06/**2.1e+06**	1.0e+05/**3.3e+04**	1.1e+09/**2.9e+07**
DomTree	8.3e+08/**5.4e+08**	**1.2e+09**/2.2e+09	1.2e+07/**4.2e+06**	**1.0e+05**/1.4e+05	1.1e+06/**5.0e+04**	**5.0e+03**/3.0e+04	5.7e+07/**5.4e+06**
KDTree	9.1e+08/**7.1e+08**	**3.1e+09**/4.0e+09	1.4e+07/**1.2e+07**	**9.2e+02**/9.4e+02	**2.6e+04**/1.1e+05	9.3e+02/9.3e+02	4.4e+07/**9.5e+06**
LinearListReduction	1.5e+08/**2.5e+07**	**9.0e+07**/1.0e+08	1.5e+06/**4.6e+04**	2.3e+02/**3.7e+01**	2.7e+05/**7.0e+03**	2.4e+02/**4.1e+01**	1.2e+07/**1.1e+06**
reduceNestedList	4.3e+08/**3.7e+07**	**1.2e+07**/2.0e+07	1.9e+06/**3.0e+05**	2.2e+02/**7.3e+01**	1.1e+06/**1.7e+04**	2.2e+02/**7.5e+01**	6.9e+06/**1.1e+06**
List	**1.3e+09**/1.5e+09	**4.5e+09**/5.6e+09	9.7e+06/**3.1e+06**	**2.7e+05**/6.0e+05	**1.9e+04**/2.1e+05	**7.6e+03**/1.6e+04	4.8e+07/**3.3e+07**
MonoTree	**2.5e+08**/2.6e+08	**9.1e+08**/1.1e+09	**2.4e+05**/5.1e+05	5.0e+04/**4.4e+04**	**3.4e+03**/1.3e+04	**1.5e+03**/2.6e+03	2.0e+06/**1.6e+06**
ObjectGraph	**8.4e+08**/8.9e+08	**3.0e+09**/4.0e+09	9.6e+06/**2.5e+06**	**1.9e+05**/2.4e+05	**3.1e+04**/5.4e+04	**1.3e+04**/1.6e+04	2.4e+07/**8.1e+06**
PiecewiseFunctions	1.2e+09/**1.1e+09**	**2.8e+09**/4.7e+09	1.4e+07/**5.3e+06**	4.8e+05/**4.2e+05**	1.8e+05/**9.1e+04**	**2.3e+04**/4.7e+04	4.5e+07/**1.0e+07**
TernaryTree	4.3e+08/**3.6e+08**	**1.2e+09**/1.5e+09	**4.4e+05**/1.6e+06	**6.5e+04**/1.2e+05	**4.6e+03**/1.9e+04	4.2e+03/**1.8e+03**	3.2e+06/**2.3e+06**
Trie	6.5e+08/**5.7e+08**	**1.2e+09**/2.2e+09	8.8e+06/**2.4e+06**	2.5e+05/**2.2e+05**	1.7e+05/**4.6e+04**	**3.4e+03**/3.8e+04	2.6e+07/**4.1e+06**

Table 4: Baseline Gibbon comparison (times only). Shown variants: AoS-mut, AoS-imm, SoA-mut. Times are full executable end-to-end wall time per run (s), not pass-sum time. **Bold** marks the fastest time across shown variants.

Program	**Am**	**Ai**	**Sm**
Compiler	29.96	110.1	**23.17**
DBQuery	26.76	33.48	**26.19**
DecisionTree	372.6	558.0	**294.7**
DomTree	20.32	34.72	**15.15**
KDTree	18.32	24.81	**14.45**
LinearListReduction	3.280	13.96	**0.8972**
reduceNestedList	16.00	23.10	**8.181**
List	**39.87**	*OOM*	47.20
MonoTree	**6.833**	14.41	7.045
ObjectGraph	**24.41**	49.26	25.31
OctTree	**63.08**	68.61	69.30
PiecewiseFunctions	39.07	57.06	**37.38**
TernaryTree	12.06	15.15	**10.98**
Trie	22.74	30.73	**21.47**
ColorOctree	9.194	12.14	**8.180**

Table 5: Baseline Gibbon comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut. Speedups are computed from full executable end-to-end wall time per run. >1× means the denominator is faster.

Program	Am/Sm	Ai/Am	Ai/Sm
Compiler	**1.29**×	**3.67**×	**4.75**×
DBQuery	1.02×	**1.25**×	**1.28**×
DecisionTree	**1.26**×	**1.50**×	**1.89**×
DomTree	**1.34**×	**1.71**×	**2.29**×
KDTree	**1.27**×	**1.35**×	**1.72**×
LinearListReduction	**3.66**×	**4.26**×	**15.56**×
reduceNestedList	**1.96**×	**1.44**×	**2.82**×
List	0.84×	–	–
MonoTree	0.97×	**2.11**×	**2.05**×
ObjectGraph	0.96×	**2.02**×	**1.95**×
OctTree	0.91×	1.09×	0.99×
PiecewiseFunctions	1.05×	**1.46**×	**1.53**×
TernaryTree	1.10×	**1.26**×	**1.38**×
Trie	1.06×	**1.35**×	**1.43**×
ColorOctree	**1.12**×	**1.32**×	**1.48**×

Table 6: Per-pass performance for `OctTree`, OctTree SoA uses 9 buffers; ColorOctree SoA uses 18 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
barnesHutPotential	F	11/16	31%	0.0416±0.00003	0.0576±0.00010	**0.0391±0.00017**	1.06×	**1.38**×	**1.47**×
countActive	F	10/16	38%	0.0356±0.00002	0.0441±0.00122	**0.0291±0.00003**	**1.23**×	**1.24**×	**1.52**×
countParticles	F	8/16	50%	0.0359±0.00002	0.0434±0.00163	**0.0264±0.00159**	**1.36**×	**1.21**×	**1.64**×
fmmPotential	F	12/16	25%	0.0967±0.00131	0.0967±0.00118	**0.0955±0.00020**	1.01×	1.00×	1.01×
sumEnergy	F	12/16	25%	0.0369±0.00003	0.0524±0.00009	**0.0328±0.00002**	**1.13**×	**1.42**×	**1.60**×
sumMass	F	10/16	38%	0.0342±0.00110	0.0433±0.00008	**0.0270±0.00002**	**1.27**×	**1.27**×	**1.61**×
clearFlags	M	15/16	6%	0.1531±0.00020	**0.1479±0.00020**	0.1725±0.00128	0.89×	0.97×	0.86×
scaleEnergy	M	16/16	0%	**0.1498±0.00103**	0.1515±0.00029	0.1756±0.00165	0.85×	1.01×	0.86×
paletteEntriesQuantized	F	13/25	48%	0.0415±0.00086	0.0531±0.00079	**0.0311±0.00005**	**1.33**×	**1.28**×	**1.71**×
quantizationErrorProxy	F	10/25	60%	0.0462±0.00001	0.0641±0.00002	**0.0427±0.00102**	1.08×	**1.39**×	**1.50**×
Total				**0.6715**	0.7541	0.6716	1.00×	**1.12**×	**1.12**×
Geomean						**1.11**×	**1.21**×	**1.34**×	

Table 7: Per-pass PAPI counters for `OctTree` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
barnesHutPotential	F	2.1e+08/**2.0e+08**	**8.2e+08**/1.0e+09	**3.7e+04**/5.4e+05	1.5e+02/**7.0e+01**	**1.8e+03**/2.3e+04	1.5e+02/**7.1e+01**	6.2e+06/**2.2e+06**
countActive	F	1.8e+08/**1.5e+08**	5.8e+08/**5.8e+08**	1.0e+06/**3.9e+05**	1.3e+02/**5.4e+01**	**3.7e+03**/1.4e+04	1.3e+02/**5.5e+01**	6.1e+06/**3.3e+05**
countParticles	F	1.8e+08/**1.3e+08**	5.7e+08/**5.7e+08**	8.9e+05/**2.1e+05**	3.4e+02/**5.1e+01**	**3.9e+03**/7.7e+03	3.4e+02/**5.3e+01**	6.0e+06/**6.3e+04**
fmmPotential	F	4.9e+08/**4.8e+08**	**1.6e+09**/1.8e+09	**2.4e+04**/6.3e+05	**4.7e+02**/5.3e+02	**1.7e+03**/2.5e+04	**4.7e+02**/5.2e+02	4.5e+06/**2.2e+06**
sumEnergy	F	1.9e+08/**1.7e+08**	**7.7e+08**/8.9e+08	**3.3e+05**/4.8e+05	2.5e+02/**8.4e+01**	**2.2e+03**/2.4e+04	2.6e+02/**8.9e+01**	6.2e+06/**2.6e+06**
sumMass	F	1.7e+08/**1.4e+08**	**5.7e+08**/6.3e+08	7.5e+05/**1.8e+05**	3.9e+02/**5.8e+01**	**2.7e+03**/8.1e+03	3.9e+02/**5.8e+01**	6.1e+06/**1.2e+06**
clearFlags	M	**3.7e+08**/4.2e+08	**8.0e+08**/1.7e+09	**8.2e+05**/2.2e+06	**1.8e+05**/5.4e+05	**8.7e+03**/3.4e+04	**8.1e+03**/1.2e+04	5.3e+06/**2.6e+06**
scaleEnergy	M	**3.6e+08**/4.4e+08	**9.1e+08**/1.9e+09	**1.2e+06**/2.5e+06	**2.4e+05**/4.3e+05	**8.8e+03**/3.7e+04	**7.9e+03**/1.6e+04	5.3e+06/**2.6e+06**
paletteEntriesQuantized	F	2.1e+08/**1.6e+08**	**6.7e+08**/7.6e+08	6.2e+06/**1.8e+06**	2.9e+02/**2.4e+02**	**9.1e+03**/4.2e+04	2.9e+02/**2.4e+02**	9.5e+06/**1.3e+06**
quantizationErrorProxy	F	2.3e+08/**2.2e+08**	**9.7e+08**/1.2e+09	4.3e+06/**9.1e+05**	1.4e+02/**1.3e+02**	**1.7e+04**/3.2e+04	1.4e+02/**1.4e+02**	9.3e+06/**2.9e+06**
Total		2.6e+09/2.5e+09	8.2e+09/1.1e+10	1.6e+07/9.8e+06	4.2e+05/9.8e+05	5.9e+04/2.5e+05	1.8e+04/3.0e+04	6.4e+07/1.8e+07

Table 8: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
blockCountPass	F	2/9	78%	0.0120±0.00001	0.0789±0.00002	**0.0019±0.00000**	**6.24**×	**6.60**×	**41.16**×
branchStatsPass	F	2/9	78%	0.0116±0.00042	0.0790±0.00002	**0.0036±0.00000**	**3.25**×	**6.82**×	**22.20**×
castInstCountPass	F	3/9	67%	0.0181±0.00002	0.0789±0.00100	**0.0013±0.00000**	**14.07**×	**4.35**×	**61.17**×
has cycle	F	4/9	56%	0.0242±0.00011	0.0923±0.00104	**0.0170±0.00015**	**1.43**×	**3.81**×	**5.43**×
instCountPass	F	2/9	78%	0.0120±0.00042	0.0791±0.00018	**0.0019±0.00000**	**6.22**×	**6.60**×	**41.04**×
latencyModelPass	F	3/9	67%	0.0119±0.00001	0.0795±0.00155	**0.0030±0.00000**	**3.99**×	**6.69**×	**26.73**×
memoryOpStatsPass	F	3/9	67%	0.0117±0.00001	0.0789±0.00002	**0.0038±0.00000**	**3.07**×	**6.73**×	**20.63**×
throughputModelPass	F	3/9	67%	0.0119±0.00001	0.0794±0.00002	**0.0030±0.00000**	**4.00**×	**6.65**×	**26.62**×
verifyIR	F	9/9	0%	0.0124±0.00010	0.0705±0.00006	**0.0034±0.00004**	**3.60**×	**5.69**×	**20.49**×
stripSideEffectsPass	M	7/9	22%	**0.0804±0.00128**	0.1855±0.00076	0.0874±0.00082	0.92×	**2.31**×	**2.12**×
targetReturnPass	M	9/9	0%	**0.0785±0.00015**	0.1841±0.00097	0.0880±0.00122	0.89×	**2.34**×	**2.09**×
Total				0.2847	1.086	**0.2143**	**1.33**×	**3.82**×	**5.07**×
Geomean						**3.21**×	**4.97**×	**15.92**×	

Table 9: Per-pass PAPI counters for `Compiler` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
blockCountPass	F	6.1e+07/**9.7e+06**	5.1e+07/**4.5e+07**	1.3e+06/**1.6e+04**	3.5e+02/**1.9e+01**	2.2e+05/**1.1e+03**	3.6e+02/**1.9e+01**	4.7e+06/**4.5e+03**
branchStatsPass	F	5.9e+07/**1.8e+07**	**7.9e+07**/8.6e+07	8.4e+05/**2.0e+04**	2.7e+02/**4.3e+01**	5.0e+04/**3.3e+03**	2.7e+02/**4.5e+01**	4.6e+06/**4.2e+05**
castInstCountPass	F	9.2e+07/**6.6e+06**	5.1e+07/**3.9e+07**	**1.3e+04**/2.3e+04	8.5e+01/**1.7e+01**	**8.1e+02**/1.1e+03	8.7e+01/**1.7e+01**	4.6e+06/**1.0e+03**
has cycle	F	1.2e+08/**8.6e+07**	**1.1e+08**/1.5e+08	2.4e+06/**2.8e+05**	4.0e+02/**1.1e+02**	1.5e+05/**2.2e+04**	4.1e+02/**1.1e+02**	5.1e+06/**2.4e+06**
instCountPass	F	6.1e+07/**9.8e+06**	**5.6e+07**/5.7e+07	1.3e+06/**1.3e+04**	2.4e+02/**1.8e+01**	2.3e+05/**1.1e+03**	2.4e+02/**1.8e+01**	4.7e+06/**4.5e+03**
latencyModelPass	F	6.0e+07/**1.5e+07**	**6.2e+07**/6.9e+07	1.2e+06/**2.9e+04**	3.6e+02/**4.0e+01**	2.0e+05/**3.9e+03**	3.7e+02/**4.2e+01**	4.7e+06/**5.3e+05**
memoryOpStatsPass	F	5.9e+07/**1.9e+07**	**8.5e+07**/9.1e+07	6.7e+05/**3.7e+04**	3.4e+02/**3.7e+01**	3.1e+04/**3.3e+03**	3.5e+02/**4.1e+01**	4.6e+06/**4.5e+05**
throughputModelPass	F	6.1e+07/**1.5e+07**	**6.2e+07**/6.9e+07	1.2e+06/**3.0e+04**	3.0e+02/**3.9e+01**	2.0e+05/**3.5e+03**	3.0e+02/**3.9e+01**	4.7e+06/**5.5e+05**
verifyIR	F	6.3e+07/**1.7e+07**	**7.4e+07**/8.6e+07	9.2e+05/**1.8e+04**	2.4e+02/**1.1e+02**	1.4e+05/**4.0e+03**	2.4e+02/**1.1e+02**	4.6e+06/**4.6e+05**
stripSideEffectsPass	M	**1.6e+08**/1.9e+08	**1.8e+08**/6.0e+08	**1.2e+06**/1.3e+06	**1.2e+05**/1.3e+05	**6.5e+03**/1.6e+04	**5.1e+03**/1.2e+04	4.1e+06/**1.2e+06**
targetReturnPass	M	**1.5e+08**/1.9e+08	**1.7e+08**/6.3e+08	2.0e+06/**1.4e+06**	**9.1e+04**/1.4e+05	**5.9e+03**/2.4e+04	**2.3e+03**/2.2e+04	4.2e+06/**1.7e+06**
Total		9.4e+08/5.8e+08	9.9e+08/1.9e+09	1.3e+07/3.1e+06	2.1e+05/2.7e+05	1.2e+06/8.4e+04	1.0e+04/3.4e+04	5.1e+07/7.7e+06

Table 10: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Am/Sm**	**Ai/Am**	**Ai/Sm**
countJoins	F	3/15	80%	0.0189±0.00001	0.0283±0.00001	**0.0109±0.00000**	**1.73×**	**1.50×**	**2.59×**
filterSelectivitySkew	F	5/15	67%	0.0189±0.00001	0.0284±0.00001	**0.0122±0.00001**	**1.55×**	**1.50×**	**2.33×**
hashJoinPressure	F	5/15	67%	0.0196±0.00001	0.0324±0.00001	**0.0125±0.00001**	**1.57×**	**1.65×**	**2.60×**
sumCost	F	6/15	60%	0.0190±0.00011	0.0282±0.00015	**0.0143±0.00009**	**1.33×**	**1.48×**	**1.97×**
sumMemory	F	6/15	60%	0.0190±0.00089	0.0284±0.00001	**0.0140±0.00002**	**1.36×**	**1.49×**	**2.03×**
sumRows	F	6/15	60%	0.0263±0.00001	0.0297±0.00002	**0.0213±0.00004**	**1.23×**	**1.13×**	**1.40×**
clearQueryFlags	M	14/15	7%	**0.0681±0.00010**	0.0755±0.00009	0.0820±0.00005	0.83×	**1.11×**	0.92×
scaleCosts	M	15/15	0%	**0.0681±0.00010**	0.0759±0.00010	0.0831±0.00004	0.82×	**1.12×**	0.91×
Total				0.2579	0.3267	**0.2503**	1.03×	**1.27×**	**1.31×**
Geomean						**1.26×**	**1.36×**	**1.71×**	

Table 11: Per-pass PAPI counters for `DBQuery` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Am/Sm)**	**INS (Am/Sm)**	**L1D (Am/Sm)**	**L1I (Am/Sm)**	**L2D (Am/Sm)**	**L2I (Am/Sm)**	**LLC (Am/Sm)**
countJoins	F	9.6e+07/**5.5e+07**	7.4e+07/**7.1e+07**	7.0e+05/**2.6e+04**	5.8e+02/**6.4e+01**	1.7e+05/**2.1e+03**	5.7e+02/**4.5e+01**	2.7e+06/**2.9e+03**
filterSelectivitySkew	F	9.5e+07/**6.2e+07**	**1.0e+08**/1.1e+08	5.3e+05/**7.5e+04**	1.9e+02/**9.0e+01**	1.2e+05/**6.3e+03**	1.9e+02/**9.4e+01**	2.6e+06/**2.3e+05**
hashJoinPressure	F	1.0e+08/**6.3e+07**	**8.4e+07**/9.9e+07	7.2e+05/**9.5e+04**	5.5e+02/**7.7e+01**	1.8e+05/**7.3e+03**	5.5e+02/**7.7e+01**	2.6e+06/**1.3e+05**
sumCost	F	9.6e+07/**7.2e+07**	**9.0e+07**/1.2e+08	6.9e+05/**8.3e+04**	1.4e+02/**1.1e+02**	1.6e+05/**1.1e+04**	1.4e+02/**1.1e+02**	2.7e+06/**3.3e+05**
sumMemory	F	9.6e+07/**7.1e+07**	**9.0e+07**/1.2e+08	5.8e+05/**8.3e+04**	3.2e+02/**6.5e+01**	1.3e+05/**1.1e+04**	3.2e+02/**6.6e+01**	2.6e+06/**3.2e+05**
sumRows	F	1.3e+08/**1.1e+08**	**2.1e+08**/2.3e+08	2.5e+05/**1.1e+05**	1.7e+02/**7.0e+01**	2.3e+04/**7.7e+03**	1.7e+02/**7.1e+01**	2.3e+06/**2.7e+05**
clearQueryFlags	M	**1.6e+08**/2.2e+08	**1.9e+08**/7.2e+08	**3.0e+05**/1.2e+06	**6.5e+04**/1.5e+05	4.2e+04/**2.3e+04**	**2.6e+03**/7.6e+03	2.5e+06/**9.1e+05**
scaleCosts	M	**1.6e+08**/2.2e+08	**2.2e+08**/7.5e+08	**2.9e+05**/1.1e+06	**6.6e+04**/1.7e+05	3.2e+04/**2.4e+04**	**2.7e+03**/1.1e+04	2.5e+06/**1.1e+06**
Total		9.4e+08/8.7e+08	1.1e+09/2.2e+09	4.1e+06/2.8e+06	1.3e+05/3.2e+05	8.5e+05/9.4e+04	7.2e+03/1.9e+04	2.1e+07/3.3e+06

Table 12: Per-pass performance for `DecisionTree`, ADT has 7 fields, SoA uses 6 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Am/Sm**	**Ai/Am**	**Ai/Sm**
classify Batch	F	5/7	29%	3.197±0.0067	4.220±0.0067	**2.593±0.0110**	**1.23×**	**1.32×**	**1.63×**
classify Depth	F	4/7	43%	0.0456±0.00001	0.0598±0.00002	**0.0366±0.00006**	**1.25×**	**1.31×**	**1.64×**
classify tree	F	5/7	29%	0.0456±0.00001	0.0598±0.00001	**0.0365±0.00005**	**1.25×**	**1.31×**	**1.64×**
countFeatureUses	F	3/7	57%	0.0492±0.00001	0.1416±0.00179	**0.0319±0.00002**	**1.54×**	**2.88×**	**4.44×**
countLeaves	F	2/7	71%	0.0466±0.00002	0.1391±0.00144	**0.0249±0.00001**	**1.87×**	**2.99×**	**5.58×**
countNodes	F	2/7	71%	0.0464±0.00098	0.1391±0.00144	**0.0261±0.00098**	**1.78×**	**3.00×**	**5.34×**
countSmallLeaves	F	3/7	57%	0.0496±0.00001	0.1358±0.00154	**0.0319±0.00003**	**1.55×**	**2.74×**	**4.25×**
inferenceCost	F	2/7	71%	0.0494±0.00001	0.1381±0.00004	**0.0286±0.00000**	**1.72×**	**2.80×**	**4.82×**
sumImpurity	F	3/7	57%	0.0489±0.00117	0.1380±0.00188	**0.0316±0.00003**	**1.55×**	**2.82×**	**4.36×**
sumPathLengths	F	3/7	57%	0.0509±0.00090	0.1371±0.00185	**0.0336±0.00015**	**1.51×**	**2.70×**	**4.08×**
sumSamples	F	3/7	57%	0.0468±0.00115	0.1332±0.00004	**0.0320±0.00002**	**1.46×**	**2.85×**	**4.16×**
treeDepth	F	2/7	71%	0.0496±0.00027	0.1379±0.00130	**0.0279±0.00001**	**1.78×**	**2.78×**	**4.95×**
Total				3.726	5.579	**2.934**	**1.27×**	**1.50×**	**1.90×**
Geomean						**1.53×**	**2.34×**	**3.57×**	

Table 13: Per-pass PAPI counters for `DecisionTree` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Am/Sm)**	**INS (Am/Sm)**	**L1D (Am/Sm)**	**L1I (Am/Sm)**	**L2D (Am/Sm)**	**L2I (Am/Sm)**	**LLC (Am/Sm)**
classify Batch	F	1.6e+10/**1.3e+10**	**3.3e+10**/5.9e+10	4.6e+08/**2.7e+07**	9.1e+04/**3.1e+04**	**1.7e+06**/2.0e+06	9.0e+04/**2.8e+04**	9.3e+08/**1.7e+07**
classify Depth	F	2.3e+08/**1.9e+08**	**4.7e+08**/8.5e+08	6.5e+06/**3.7e+05**	8.0e+02/**5.3e+02**	**2.0e+04**/2.8e+04	8.0e+02/**4.9e+02**	1.3e+07/**2.8e+05**
classify tree	F	2.3e+08/**1.9e+08**	**4.7e+08**/8.5e+08	6.5e+06/**3.7e+05**	1.3e+03/**5.4e+02**	**2.2e+04**/2.8e+04	1.3e+03/**5.0e+02**	1.3e+07/**2.8e+05**
countFeatureUses	F	2.5e+08/**1.6e+08**	**7.5e+08**/7.8e+08	7.6e+06/**2.6e+05**	1.9e+03/**6.4e+02**	7.3e+04/**1.3e+04**	1.9e+03/**6.4e+02**	1.5e+07/**2.0e+06**
countLeaves	F	2.4e+08/**1.3e+08**	**6.0e+08**/6.1e+08	7.3e+06/**1.1e+05**	7.6e+02/**3.7e+02**	4.7e+04/**7.9e+03**	7.6e+02/**3.7e+02**	1.4e+07/**3.3e+05**
countNodes	F	2.4e+08/**1.3e+08**	**6.1e+08**/6.2e+08	7.1e+06/**1.1e+05**	6.7e+02/**2.4e+02**	4.7e+04/**7.9e+03**	6.7e+02/**2.4e+02**	1.4e+07/**3.5e+05**
countSmallLeaves	F	2.5e+08/**1.6e+08**	**7.1e+08**/8.4e+08	5.9e+06/**8.8e+04**	1.4e+03/**4.7e+02**	3.1e+04/**1.3e+04**	1.4e+03/**4.7e+02**	1.5e+07/**2.0e+06**
inferenceCost	F	2.5e+08/**1.5e+08**	6.1e+08/**6.1e+08**	8.1e+06/**1.2e+05**	1.3e+03/**3.3e+02**	2.2e+04/**7.7e+03**	1.3e+03/**3.3e+02**	1.4e+07/**3.3e+05**
sumImpurity	F	2.5e+08/**1.6e+08**	**7.6e+08**/7.8e+08	7.6e+06/**5.7e+05**	9.2e+02/**5.1e+02**	7.3e+04/**1.3e+04**	9.1e+02/**5.1e+02**	1.5e+07/**2.0e+06**
sumPathLengths	F	2.6e+08/**1.7e+08**	**8.0e+08**/9.0e+08	7.8e+06/**3.9e+05**	1.2e+03/**8.7e+02**	8.9e+04/**1.4e+04**	1.2e+03/**8.7e+02**	1.5e+07/**2.0e+06**
sumSamples	F	2.4e+08/**1.6e+08**	**6.4e+08**/7.7e+08	6.2e+06/**2.5e+05**	9.0e+02/**5.8e+02**	4.1e+04/**1.4e+04**	8.7e+02/**5.8e+02**	1.5e+07/**2.0e+06**
treeDepth	F	2.5e+08/**1.4e+08**	6.1e+08/**6.1e+08**	8.1e+06/**1.1e+05**	1.5e+03/**3.9e+02**	2.1e+04/**7.8e+03**	1.5e+03/**3.9e+02**	1.4e+07/**3.3e+05**
Total		1.9e+10/1.5e+10	4.0e+10/6.8e+10	5.4e+08/2.9e+07	1.0e+05/3.6e+04	2.2e+06/2.1e+06	1.0e+05/3.3e+04	1.1e+09/2.9e+07

Table 14: Per-pass performance for `DomTree`, ADT has 15 fields, SoA uses 14 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Am/Sm**	**Ai/Am**	**Ai/Sm**
count styled	F	3/15	80%	0.0344±0.00002	0.0692±0.00015	**0.0116±0.00001**	**2.96×**	**2.01×**	**5.96×**
find max Bottom	F	5/15	67%	0.0344±0.00002	0.0688±0.00002	**0.0190±0.00003**	**1.81×**	**2.00×**	**3.61×**
SumArea	F	6/15	60%	0.0356±0.00002	0.0696±0.00006	**0.0191±0.00003**	**1.86×**	**1.96×**	**3.64×**
sumTextWidth	F	3/15	80%	0.0342±0.00001	0.0697±0.00001	**0.0117±0.00001**	**2.93×**	**2.04×**	**5.98×**
computeWidths	M	14/15	7%	**0.0295±0.00004**	0.0302±0.00130	0.0443±0.00006	0.67×	1.02×	0.68×
scaleLayout	M	15/15	0%	**0.0288±0.00015**	0.0301±0.00131	0.0354±0.00085	0.81×	1.04×	0.85×
Total				0.1969	0.3377	**0.1412**	**1.39×**	**1.72×**	**2.39×**
Geomean						**1.58×**	**1.61×**	**2.54×**	

Table 15: Per-pass PAPI counters for `DomTree` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Am/Sm)**	**INS (Am/Sm)**	**L1D (Am/Sm)**	**L1I (Am/Sm)**	**L2D (Am/Sm)**	**L2I (Am/Sm)**	**LLC (Am/Sm)**
count styled	F	1.7e+08/**5.9e+07**	**2.6e+08**/2.6e+08	2.2e+06/**3.3e+04**	1.1e+03/**9.6e+01**	3.0e+05/**4.9e+03**	1.0e+03/**9.6e+01**	1.4e+07/**6.4e+05**
find max Bottom	F	1.7e+08/**9.6e+07**	**2.5e+08**/4.1e+08	2.4e+06/**1.2e+06**	4.8e+02/**1.4e+02**	1.5e+05/**9.4e+03**	4.8e+02/**1.4e+02**	1.4e+07/**1.4e+06**
SumArea	F	1.8e+08/**9.7e+07**	**2.9e+08**/4.9e+08	2.8e+06/**5.3e+05**	3.1e+02/**9.9e+01**	1.7e+05/**1.2e+04**	3.1e+02/**9.9e+01**	1.4e+07/**1.9e+06**
sumTextWidth	F	1.7e+08/**5.9e+07**	**2.0e+08**/2.4e+08	2.5e+06/**2.0e+04**	6.4e+02/**1.0e+02**	4.4e+05/**4.2e+03**	6.4e+02/**1.0e+02**	1.4e+07/**6.5e+05**
computeWidths	M	**6.4e+07**/1.4e+08	**1.4e+08**/4.6e+08	**1.3e+06**/1.6e+06	**3.7e+04**/4.4e+04	**6.0e+03**/1.1e+04	**1.3e+03**/1.5e+04	1.2e+06/**4.2e+05**
scaleLayout	M	**5.9e+07**/9.1e+07	**8.8e+07**/2.9e+08	1.0e+06/**7.8e+05**	**6.2e+04**/9.1e+04	1.5e+04/**8.1e+03**	**1.3e+03**/1.4e+04	1.3e+06/**4.5e+05**
Total		8.3e+08/5.4e+08	1.2e+09/2.2e+09	1.2e+07/4.2e+06	1.0e+05/1.4e+05	1.1e+06/5.0e+04	5.0e+03/3.0e+04	5.7e+07/5.4e+06

Table 16: Per-pass performance for `KDTree`, ADT has 17 fields, SoA uses 16 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
countInRange tight_box	F	13/17	24%	0.0282±0.00001	0.0403±0.00001	**0.0198±0.00001**	**1.43×**	**1.43×**	**2.04×**
Find nearest Neighbour	F	13/17	24%	0.0281±0.00002	0.0397±0.00006	**0.0213±0.00004**	**1.32×**	**1.41×**	**1.86×**
photonMappingBenchmark	F	12/17	29%	0.0421±0.00099	0.0476±0.00002	**0.0413±0.00108**	1.02×	**1.13×**	**1.15×**
pointCloudNeighborhood	F	11/17	35%	0.0250±0.00001	0.0389±0.00084	**0.0166±0.00089**	**1.50×**	**1.56×**	**2.34×**
sumMassInRange	F	14/17	18%	0.0278±0.00081	0.0404±0.00001	**0.0199±0.00001**	**1.40×**	**1.45×**	**2.03×**
twoPointCorrelation bin_8_16	F	11/17	35%	0.0290±0.00002	0.0393±0.00001	**0.0210±0.00000**	**1.38×**	**1.36×**	**1.87×**
Total				0.1802	0.2462	**0.1399**	**1.29×**	**1.37×**	**1.76×**
Geomean						**1.33×**	**1.38×**	**1.84×**	

Table 17: Per-pass PAPI counters for `KDTree` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
countInRange tight_box	F	1.4e+08/**1.0e+08**	**4.0e+08**/5.5e+08	**1.9e+06**/2.2e+06	2.0e+02/**1.1e+02**	**3.8e+03**/1.9e+04	2.0e+02/**1.1e+02**	7.7e+06/**1.7e+06**
Find nearest Neighbour	F	1.4e+08/**1.1e+08**	**4.7e+08**/6.3e+08	2.7e+06/**1.9e+06**	1.2e+02/**9.4e+01**	**9.0e+03**/2.0e+04	1.2e+02/**9.4e+01**	7.7e+06/**1.7e+06**
photonMappingBenchmark	F	2.1e+08/**2.1e+08**	**1.0e+09**/1.2e+09	**2.2e+06**/2.4e+06	**2.4e+02**/4.1e+02	**3.5e+03**/1.6e+04	**2.4e+02**/4.1e+02	5.6e+06/**1.5e+06**
pointCloudNeighborhood	F	1.3e+08/**8.4e+07**	**3.3e+08**/4.8e+08	2.6e+06/**1.3e+06**	**8.7e+01**/1.2e+02	**3.4e+03**/1.6e+04	**8.9e+01**/1.2e+02	7.8e+06/**1.5e+06**
sumMassInRange	F	1.4e+08/**1.0e+08**	**4.0e+08**/5.7e+08	**2.1e+06**/3.7e+06	**1.3e+02**/1.5e+02	**3.6e+03**/2.6e+04	**1.3e+02**/1.5e+02	7.7e+06/**1.9e+06**
twoPointCorrelation bin_8_16	F	1.5e+08/**1.1e+08**	**4.8e+08**/6.1e+08	2.1e+06/**7.3e+05**	1.5e+02/**5.6e+01**	**2.8e+03**/1.5e+04	1.5e+02/**5.4e+01**	7.7e+06/**1.3e+06**
Total		9.1e+08/7.1e+08	3.1e+09/4.0e+09	1.4e+07/1.2e+07	9.2e+02/9.4e+02	2.6e+04/1.1e+05	9.3e+02/9.3e+02	4.4e+07/9.5e+06

Table 18: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
reduction	F	2/11	82%	0.0298±0.00002	0.1318±0.00188	**0.0049±0.00009**	**6.03×**	**4.42×**	**26.67×**
Total				0.0298	0.1318	**0.0049**	**6.03×**	**4.42×**	**26.67×**
Geomean						**6.03×**	**4.42×**	**26.67×**	

Table 19: Per-pass PAPI counters for `LinearListReduction` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
reduction	F	1.5e+08/**2.5e+07**	**9.0e+07**/1.0e+08	1.5e+06/**4.6e+04**	2.3e+02/**3.7e+01**	2.7e+05/**7.0e+03**	2.4e+02/**4.1e+01**	1.2e+07/**1.1e+06**
Total		1.5e+08/2.5e+07	9.0e+07/1.0e+08	1.5e+06/4.6e+04	2.3e+02/3.7e+01	2.7e+05/7.0e+03	2.4e+02/4.1e+01	1.2e+07/1.1e+06

Table 20: Per-pass performance for `reduceNestedList`, ADT has 6 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
reduction	F	2/6	67%	0.0853±0.00062	0.0929±0.00086	**0.0074±0.00001**	**11.58×**	1.09×	**12.61×**
Total				0.0853	0.0929	**0.0074**	**11.58×**	1.09×	**12.61×**
Geomean						**11.58×**	1.09×	**12.61×**	

Table 21: Per-pass PAPI counters for `reduceNestedList` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
reduction	F	4.3e+08/**3.7e+07**	**1.2e+07**/2.0e+07	1.9e+06/**3.0e+05**	2.2e+02/**7.3e+01**	1.1e+06/**1.7e+04**	2.2e+02/**7.5e+01**	6.9e+06/**1.1e+06**
Total		4.3e+08/3.7e+07	1.2e+07/2.0e+07	1.9e+06/3.0e+05	2.2e+02/7.3e+01	1.1e+06/1.7e+04	2.2e+02/7.5e+01	6.9e+06/1.1e+06

Table 22: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
length List	F	1/2	50%	0.0441±0.00002	*OOM*	**0.0216±0.00001**	**2.04×**	–	–
sumList	F	2/2	0%	**0.0443±0.00002**	*OOM*	0.0464±0.00071	0.95×	–	–
sumList tail recursive	F	2/2	0%	**0.0437±0.00003**	*OOM*	0.0463±0.00002	0.94×	–	–
add1 List	M	2/2	0%	**0.2508±0.00039**	*OOM*	0.3353±0.00167	0.75×	–	–
Total				**0.3828**	–	0.4496	0.85×	–	–
Geomean						1.08×	–	–	

Table 23: Per-pass PAPI counters for `List` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
length List	F	2.2e+08/**1.1e+08**	8.0e+08/**8.0e+08**	**1.1e+05**/1.8e+05	9.8e+01/**6.6e+01**	**3.1e+03**/1.9e+04	9.9e+01/**6.4e+01**	1.3e+07/**1.2e+06**
sumList	F	**2.2e+08**/2.4e+08	**9.0e+08**/1.0e+09	5.9e+05/**5.5e+05**	1.6e+02/**5.8e+01**	**3.0e+03**/6.9e+04	1.6e+02/**6.1e+01**	1.3e+07/**1.2e+07**
sumList tail recursive	F	**2.2e+08**/2.3e+08	**9.0e+08**/1.0e+09	8.2e+05/**5.5e+05**	1.3e+02/**1.3e+02**	**3.2e+03**/7.0e+04	1.3e+02/1.3e+02	1.3e+07/**1.2e+07**
add1 List	M	**5.8e+08**/9.5e+08	**1.9e+09**/2.8e+09	8.1e+06/**1.8e+06**	**2.7e+05**/6.0e+05	**1.0e+04**/5.2e+04	**7.2e+03**/1.6e+04	9.1e+06/**7.8e+06**
Total		1.3e+09/1.5e+09	4.5e+09/5.6e+09	9.7e+06/3.1e+06	2.7e+05/6.0e+05	1.9e+04/2.1e+05	7.6e+03/1.6e+04	4.8e+07/3.3e+07

Table 24: Per-pass performance for `MonoTree`, ADT has 3 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
sumTree	F	3/3	0%	**0.0089±0.00001**	0.0400±0.00001	0.0113±0.00066	0.79×	**4.49×**	**3.56×**
sumTree TailRec	F	3/3	0%	**0.0082±0.00000**	0.0401±0.00001	0.0110±0.00073	0.75×	**4.87×**	**3.64×**
add1Tree	M	3/3	0%	0.0459±0.00144	0.0602±0.00141	**0.0422±0.00034**	1.09×	**1.31×**	**1.43×**
Total				**0.0631**	0.1403	0.0645	0.98×	**2.23×**	**2.18×**
Geomean						0.86×	**3.06×**	**2.64×**	

Table 25: Per-pass PAPI counters for `MonoTree` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
sumTree	F	**4.5e+07**/5.7e+07	**2.2e+08**/2.6e+08	**5.0e+03**/3.5e+04	1.8e+02/**1.7e+02**	**6.0e+02**/4.0e+03	1.8e+02/**1.7e+02**	7.4e+05/**5.9e+05**
sumTree TailRec	F	**4.2e+07**/5.6e+07	**1.9e+08**/2.2e+08	**6.0e+03**/2.0e+04	**2.0e+02**/3.0e+02	**7.2e+02**/3.9e+03	**2.0e+02**/3.0e+02	7.5e+05/**5.8e+05**
add1Tree	M	1.6e+08/**1.4e+08**	**5.0e+08**/5.9e+08	**2.3e+05**/4.5e+05	5.0e+04/**4.3e+04**	**2.1e+03**/4.9e+03	**1.1e+03**/2.2e+03	5.1e+05/**4.5e+05**
Total		2.5e+08/2.6e+08	9.1e+08/1.1e+09	2.4e+05/5.1e+05	5.0e+04/4.4e+04	3.4e+03/1.3e+04	1.5e+03/2.6e+03	2.0e+06/1.6e+06

Table 26: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
countLargeItems	F	3/5	40%	0.0131±0.00000	0.0442±0.00013	**0.0106±0.00002**	**1.24×**	**3.38×**	**4.18×**
countMarked	F	3/5	40%	0.0127±0.00001	0.0443±0.00001	**0.0106±0.00001**	**1.20×**	**3.49×**	**4.19×**
countSurvivors	F	4/5	20%	0.0136±0.00001	0.0443±0.00101	**0.0132±0.00001**	1.03×	**3.26×**	**3.36×**
deadBytes	F	4/5	20%	0.0133±0.00001	0.0439±0.00001	**0.0126±0.00068**	1.06×	**3.30×**	**3.49×**
liveBytes	F	4/5	20%	0.0137±0.00001	0.0433±0.00002	**0.0126±0.00001**	1.09×	**3.16×**	**3.45×**
sumObjIds	F	3/5	40%	0.0124±0.00000	0.0431±0.00001	**0.0106±0.00002**	**1.17×**	**3.47×**	**4.08×**
totalHeapSize	F	3/5	40%	0.0123±0.00009	0.0429±0.00028	**0.0107±0.00009**	**1.16×**	**3.48×**	**4.03×**
sweepUnmarked	M	5/5	0%	**0.0706±0.00007**	0.0876±0.00109	0.0809±0.00003	0.87×	**1.24×**	1.08×
touchHotObjects	M	5/5	0%	**0.0720±0.00122**	0.0868±0.00156	0.0804±0.00123	0.90×	**1.21×**	1.08×
Total				**0.2337**	0.4804	0.2419	0.97×	**2.06×**	**1.99×**
Geomean						1.07×	**2.69×**	**2.88×**	

Table 27: Per-pass PAPI counters for `ObjectGraph` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
countLargeItems	F	6.6e+07/**5.3e+07**	**2.6e+08**/2.8e+08	1.1e+06/**2.6e+05**	2.0e+02/**9.1e+01**	6.1e+03/**3.6e+03**	2.0e+02/**9.5e+01**	2.9e+06/**6.2e+05**
countMarked	F	6.4e+07/**5.3e+07**	**2.6e+08**/2.8e+08	1.2e+06/**2.5e+05**	2.2e+02/**1.0e+02**	**3.1e+03**/4.0e+03	2.2e+02/**1.0e+02**	2.9e+06/**6.3e+05**
countSurvivors	F	6.9e+07/**6.7e+07**	**3.1e+08**/3.8e+08	9.9e+05/**1.5e+05**	**2.3e+02**/2.6e+02	**2.3e+03**/5.8e+03	**2.3e+02**/2.6e+02	2.7e+06/**1.2e+06**
deadBytes	F	6.7e+07/**6.4e+07**	**2.7e+08**/3.5e+08	7.8e+05/**1.1e+05**	1.8e+02/**1.6e+02**	**2.3e+03**/5.4e+03	1.9e+02/**1.7e+02**	2.7e+06/**1.2e+06**
liveBytes	F	6.9e+07/**6.4e+07**	**2.7e+08**/3.5e+08	6.8e+05/**1.1e+05**	**1.4e+02**/1.8e+02	**1.9e+03**/5.4e+03	**1.4e+02**/1.9e+02	2.7e+06/**1.2e+06**
sumObjIds	F	6.3e+07/**5.3e+07**	**2.5e+08**/2.7e+08	1.3e+06/**8.1e+04**	**1.5e+02**/1.8e+02	**3.0e+03**/4.4e+03	**1.6e+02**/1.8e+02	2.8e+06/**6.2e+05**
totalHeapSize	F	6.2e+07/**5.4e+07**	**2.5e+08**/2.7e+08	1.4e+06/**3.4e+05**	1.9e+02/**1.2e+02**	**2.8e+03**/3.6e+03	1.9e+02/**1.2e+02**	2.9e+06/**6.1e+05**
sweepUnmarked	M	**1.9e+08**/2.4e+08	**5.5e+08**/8.8e+08	1.1e+06/**6.4e+05**	**7.2e+04**/1.1e+05	**4.2e+03**/1.1e+04	**4.2e+03**/9.1e+03	2.0e+06/**1.0e+06**
touchHotObjects	M	**1.9e+08**/2.4e+08	**5.7e+08**/9.0e+08	1.0e+06/**5.5e+05**	**1.1e+05**/1.2e+05	**5.1e+03**/1.1e+04	7.2e+03/**6.2e+03**	2.0e+06/**1.0e+06**
Total		8.4e+08/8.9e+08	3.0e+09/4.0e+09	9.6e+06/2.5e+06	1.9e+05/2.4e+05	3.1e+04/5.4e+04	1.3e+04/1.6e+04	2.4e+07/8.1e+06

Table 28: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
autorefineMaxLevel	F	4/8	50%	**0.0203±0.00001**	0.0450±0.00001	0.0214±0.00101	0.95×	**2.22×**	**2.10×**
compressMass	F	3/8	62%	0.0193±0.00000	0.0453±0.00097	**0.0117±0.00001**	**1.65×**	**2.35×**	**3.89×**
lbDeuxLoadProxy	F	5/8	38%	0.0212±0.00001	0.0470±0.00001	**0.0190±0.00001**	**1.12×**	**2.22×**	**2.48×**
norm2Estimate	F	5/8	38%	0.0246±0.00006	0.0545±0.00051	**0.0184±0.00005**	**1.33×**	**2.22×**	**2.95×**
pmapCutHistogram	F	4/8	50%	0.0315±0.00090	0.0555±0.00002	**0.0134±0.00001**	**2.35×**	**1.76×**	**4.14×**
truncateTolViolations	F	3/8	62%	0.0196±0.00000	0.0458±0.00001	**0.0117±0.00001**	**1.68×**	**2.33×**	**3.92×**
addConstPW	M	8/8	0%	**0.1189±0.00014**	0.1293±0.00021	0.1305±0.00081	0.91×	1.09×	0.99×
diffPW	M	8/8	0%	**0.1187±0.00126**	0.1320±0.00013	0.1292±0.00015	0.92×	**1.11×**	1.02×
Total				0.3741	0.5545	**0.3553**	1.05×	**1.48×**	**1.56×**
Geomean						**1.29×**	**1.83×**	**2.37×**	

Table 29: Per-pass PAPI counters for `PiecewiseFunctions` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
autorefineMaxLevel	F	**1.0e+08**/1.1e+08	**2.5e+08**/3.7e+08	2.5e+06/**2.2e+05**	4.9e+02/**1.3e+02**	1.1e+04/**7.0e+03**	4.9e+02/**1.4e+02**	6.1e+06/**9.3e+05**
compressMass	F	9.8e+07/**5.9e+07**	**2.5e+08**/2.8e+08	2.2e+06/**3.2e+04**	5.0e+02/**1.6e+02**	1.9e+04/**5.0e+03**	5.0e+02/**1.7e+02**	6.2e+06/**6.0e+05**
lbDeuxLoadProxy	F	1.1e+08/**9.6e+07**	**3.3e+08**/4.9e+08	1.2e+06/**3.1e+05**	9.0e+02/**2.0e+02**	1.5e+04/**1.0e+04**	9.0e+02/**2.0e+02**	6.2e+06/**1.6e+06**
norm2Estimate	F	1.2e+08/**9.4e+07**	**3.4e+08**/4.4e+08	1.2e+06/**4.7e+05**	3.2e+02/**1.4e+02**	1.1e+04/**9.4e+03**	3.2e+02/**1.4e+02**	6.2e+06/**1.5e+06**
pmapCutHistogram	F	1.6e+08/**6.8e+07**	**2.8e+08**/3.7e+08	1.6e+06/**7.7e+04**	1.4e+03/**4.4e+02**	9.1e+04/**7.6e+03**	1.4e+03/**4.4e+02**	4.7e+06/**1.2e+06**
truncateTolViolations	F	1.0e+08/**5.9e+07**	**2.5e+08**/2.9e+08	2.1e+06/**3.6e+04**	5.7e+02/**2.0e+02**	1.6e+04/**5.0e+03**	5.7e+02/**2.1e+02**	6.3e+06/**6.2e+05**
addConstPW	M	**2.7e+08**/3.3e+08	**5.7e+08**/1.2e+09	**1.2e+06**/2.1e+06	**1.9e+05**/2.0e+05	**9.8e+03**/2.5e+04	**1.3e+04**/2.7e+04	4.5e+06/**1.8e+06**
diffPW	M	**2.8e+08**/3.3e+08	**5.9e+08**/1.3e+09	**1.3e+06**/2.0e+06	2.9e+05/**2.2e+05**	**1.1e+04**/2.2e+04	**6.0e+03**/1.9e+04	4.5e+06/**1.8e+06**
Total		1.2e+09/1.1e+09	2.8e+09/4.7e+09	1.4e+07/5.3e+06	4.8e+05/4.2e+05	1.8e+05/9.1e+04	2.3e+04/4.7e+04	4.5e+07/1.0e+07

Table 30: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Am/Sm	Ai/Am	Ai/Sm
sum tree	F	5/5	0%	0.0294±0.00006	0.0501±0.00002	**0.0267±0.00007**	1.10×	**1.71×**	**1.87×**
add 1 tree	M	5/5	0%	0.0859±0.00112	0.0931±0.00172	**0.0775±0.00013**	**1.11×**	1.08×	**1.20×**
Total				0.1153	0.1432	**0.1042**	**1.11×**	**1.24×**	**1.37×**
Geomean						**1.10×**	**1.36×**	**1.50×**	

Table 31: Per-pass PAPI counters for `TernaryTree` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Am/Sm)	INS (Am/Sm)	L1D (Am/Sm)	L1I (Am/Sm)	L2D (Am/Sm)	L2I (Am/Sm)	LLC (Am/Sm)
sum tree	F	1.5e+08/**1.4e+08**	**4.6e+08**/5.4e+08	**6.5e+04**/3.5e+05	**1.1e+02**/1.7e+02	**7.8e+02**/9.0e+03	**1.2e+02**/1.6e+02	1.7e+06/**1.2e+06**
add 1 tree	M	2.8e+08/**2.3e+08**	**7.5e+08**/9.8e+08	**3.8e+05**/1.2e+06	**6.5e+04**/1.2e+05	**3.9e+03**/1.0e+04	4.1e+03/**1.7e+03**	1.5e+06/**1.2e+06**
Total		4.3e+08/3.6e+08	1.2e+09/1.5e+09	4.4e+05/1.6e+06	6.5e+04/1.2e+05	4.6e+03/1.9e+04	4.2e+03/1.8e+03	3.2e+06/2.3e+06

Table 32: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Am/Sm**	**Ai/Am**	**Ai/Sm**
autocompleteTopKProxy	F	5/10	50%	0.0144±0.00001	0.0273±0.00001	**0.0085±0.00003**	**1.69×**	**1.90×**	**3.22×**
countLazyNodes	F	4/10	60%	0.0129±0.00001	0.0277±0.00001	**0.0067±0.00056**	**1.94×**	**2.14×**	**4.15×**
countTerminals	F	3/10	70%	0.0122±0.00001	0.0258±0.00052	**0.0058±0.00001**	**2.10×**	**2.11×**	**4.43×**
sumPrefixFreq	F	3/10	70%	0.0124±0.00021	0.0263±0.00015	**0.0057±0.00006**	**2.18×**	**2.12×**	**4.60×**
sumSubtreeHints	F	3/10	70%	0.0124±0.00000	0.0262±0.00082	**0.0057±0.00000**	**2.17×**	**2.12×**	**4.61×**
decayTrieStats	M	10/10	0%	**0.0755±0.00006**	0.0798±0.00022	0.0843±0.00167	0.90×	1.06×	0.95×
resetTraversalState	M	10/10	0%	**0.0770±0.00006**	0.0809±0.00011	0.0823±0.00010	0.94×	1.05×	0.98×
Total				0.2168	0.2939	**0.1990**	1.09×	**1.36×**	**1.48×**
Geomean						**1.60×**	**1.71×**	**2.74×**	

Table 33: Per-pass PAPI counters for `Trie` (Am/Sm). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Am/Sm)**	**INS (Am/Sm)**	**L1D (Am/Sm)**	**L1I (Am/Sm)**	**L2D (Am/Sm)**	**L2I (Am/Sm)**	**LLC (Am/Sm)**
autocompleteTopKProxy	F	7.3e+07/**4.3e+07**	**1.3e+08**/2.1e+08	1.5e+06/**2.5e+05**	**5.1e+01**/5.4e+01	4.7e+04/**4.6e+03**	5.7e+01/**5.3e+01**	3.7e+06/**7.9e+05**
countLazyNodes	F	6.5e+07/**3.4e+07**	**1.3e+08**/1.6e+08	1.3e+06/**2.6e+04**	**7.9e+01**/9.5e+01	1.7e+04/**3.3e+03**	**8.6e+01**/9.3e+01	4.0e+06/**5.6e+05**
countTerminals	F	6.2e+07/**2.9e+07**	**1.0e+08**/1.2e+08	1.4e+06/**1.2e+04**	1.2e+02/**8.3e+01**	3.0e+04/**2.3e+03**	1.3e+02/**8.0e+01**	3.9e+06/**2.1e+05**
sumPrefixFreq	F	6.3e+07/**2.9e+07**	**1.1e+08**/1.2e+08	1.5e+06/**1.4e+04**	**7.3e+01**/7.5e+01	2.2e+04/**2.5e+03**	8.0e+01/**7.5e+01**	3.9e+06/**2.1e+05**
sumSubtreeHints	F	6.3e+07/**2.9e+07**	**1.1e+08**/1.2e+08	1.5e+06/**1.5e+04**	1.8e+02/**7.0e+01**	3.4e+04/**2.5e+03**	1.9e+02/**6.9e+01**	3.9e+06/**2.1e+05**
decayTrieStats	M	**1.6e+08**/2.1e+08	**3.5e+08**/7.9e+08	**9.4e+05**/1.3e+06	**1.1e+05**/1.2e+05	**7.3e+03**/1.9e+04	**1.4e+03**/2.5e+04	3.1e+06/**1.2e+06**
resetTraversalState	M	**1.6e+08**/2.0e+08	**2.8e+08**/7.0e+08	**5.8e+05**/8.6e+05	1.4e+05/**1.1e+05**	**8.4e+03**/1.1e+04	**1.5e+03**/1.2e+04	3.0e+06/**8.9e+05**
Total		6.5e+08/5.7e+08	1.2e+09/2.2e+09	8.8e+06/2.4e+06	2.5e+05/2.2e+05	1.7e+05/4.6e+04	3.4e+03/3.8e+04	2.6e+07/4.1e+06